

Hackathon Goals and Criteria

The **“Built with Python” hackathon** (student-only, deadline Jun 20, 2026 [1†L105-L110]) rewards creative, working Python projects. Judges explicitly score **Functionality & Python usage (35%), Creativity (25%), Usability/Clarity (20%), Presentation (10%), and Problem-solving/learning (10%)** [2†L287-L295] .

Submissions must include a clear explanation of what you built, how Python is used, screenshots, a code link, and a 1–3 minute demo video [2†L208-L216] . In short, the idea must be **easy to understand** yet show technical depth [10†L281-L289] . Judges “look at creativity, Python usage, functionality, presentation, and learning effort” [2†L223-L225] .

Idea Assessment – Strengths and Challenges

Your **AI “coding coach” app** is ambitious and Python-centric. It targets a clear problem (coding interview prep) with three modes: AI Q&A interview practice, “think aloud” coding explanations, and a fun “code roast.” This aligns well with the hackathon’s education and creativity theme. Using Python for AI (Groq LLMs, Whisper for speech, AST/pyflakes for code analysis) ticks the **Functionality & Python** box [2†L287-L295] . The **Creativity** is high: mock interviews and roast-your-code are novel ideas in an educational hackathon. If executed well, the project can stand out for originality and tech usage.

However, it’s very **complex** for a short student hackathon. With only ~2 days until the deadline (June 20) [1†L105-L110] , implementing three modes plus a polished UI and demo is a big task. There’s a risk of **scope creep** or incomplete features. Also, judges value clarity. As one hackathon write-up advises, “an idea must be simple enough for a child to understand” [10†L281-L289] . Jamming three AI modes together could confuse non-technical judges unless explained extremely clearly.

To win with this idea, focus on delivering a **clean, working core**. For example, you might prioritize **one or two modes** and ensure they work smoothly. *If* you can’t fully implement all modes, consider trimming the scope. You could, for example, focus on a streamlined **“interview practice” mode** or a single **code-feedback mode**, and present the others conceptually. This ensures a finished prototype, which hackathons value: as a similar project (“AI Placement Coach”) emphasizes, *“built a fully working MVP with live deployment”* as their proudest accomplishment [27†L199-L204] .

Benchmark – Similar Projects

A good analogy is the “AI Placement Coach” project, which used AI to simulate coding interviews [24†L170-L173] . It let users solve problems (and speak their answers) while an AI judged correctness and gave feedback [24†L170-L173] . They highlighted creating **structured feedback** (scores, strengths, improvement areas) just like a real interviewer [24†L170-L173] . In their summary, they boast of a *“fully working MVP with live deployment”* and an *“interview-like user experience”* [27†L199-L204] . They also stressed the extra creative touches (multimodal input, structured rubrics) and explained challenges. Key takeaways:

successful projects show a **working demo**, focus on the core user experience, and articulate what makes their solution unique.

Another example: science/AI hackathon winners often tell a clear story – define a problem, show how the project solves it, and demonstrate the result. The hackathon listing itself reminds you to explain *“what you built, how Python was used”* [2†L208-L216] . Even if you can't implement everything, have a clear narrative. For instance, a coder might say: *“The problem is students lack personalized coding practice; our app (1) asks real interview questions, (2) lets you talk through your thinking, and (3) analyzes your code quality. These three features distinguish our project.”*

Mapping Your Features to Criteria

- **Functionality & Python (35%):** Emphasize each mode's Python core. For example, mention that **Groq+Llama-3** powers question generation and answer grading, **OpenAI Whisper** handles speech-to-text, and **pyflakes/AST** catch code errors. Make sure everything you demo actually *works*. Even dummy or partial functionality with a polished UI can score points, but you should clearly explain which parts are real and which are prototype.
- **Creativity & Originality (25%):** Your blend of “interview coach”, “think-aloud grader”, and “code roaster” is unique. Highlight this novelty. The “roast” mode in particular is creative – it gamifies feedback. If there are safeguards (e.g. non-toxic roasting), mention them. Stress how this solves the problem “in a fun, engaging way.”
- **Usability & Clarity (20%):** Keep the UI and instructions extremely simple. For example, a Streamlit sidebar could let the user pick a mode and next steps. Use clear labels: *“Start Mock Interview”, “Explain Solution Out Loud”, “Paste Your Code”*. Judges should **instantly understand** how to use it. Provide a quick tutorial or on-screen guidance so anyone can try it. This also means your Devpost submission (text, screenshots, video) must be **crisp and jargon-free**. One guide advises boiling your pitch to *“a simple statement and highlight 3 key features or less.”* [31†L162-L170]
- **Presentation & Explanation (10%):** Craft a tight narrative. In your submission/demo video, start by stating the *problem* (e.g., “Preparing for coding interviews is hard!”) and then your *solution*. Structure it like: **Problem → Demo of solution → What's unique**. When describing your build, explicitly answer the hackathon checklist: *what it does, how Python is used* [2†L208-L216] . For example, “We used Whisper (Python) to transcribe speech and call a GPT model for feedback.” Use the required media: add at least one screenshot of the app, and a short demo video showing someone using each mode.
- **Problem Solving & Learning (10%):** Be honest about challenges. In your write-up, mention any hurdles (e.g. integrating speech, tuning AI prompts) and how you overcame or would overcome them. Judges look for evidence of learning from the process [2†L220-L225] . For instance, you might say “We learned how to restrict the AI's tone for constructive criticism.” This shows growth and effort.

Development Plan & MVP Roadmap

With the deadline imminent, plan your hours wisely. Here's a rough plan:

1. **Core MVP Selection (Next few hours):** Decide your minimal viable feature set. You might choose to implement **Interview Mode fully first**, since that showcases Python/AI clearly, and add a basic Code Roast if time permits. (Or vice versa, pick the most impressive one.) Make clear in your pitch that the other modes are planned features.
2. **Backend Logic (Day 1):** Develop each mode's logic in isolation. For Interview: hard-code a small question bank and use a simple Python function (or a small LLM) to check answers. For Think-Aloud: pick one common DSA problem; allow the user to submit their explanation (text input as backup to speech). For Code Roast: integrate `pyflakes` /AST to catch errors and craft some "roast" prompts. Ensure Python modules (e.g. `sounddevice`, `scipy`, `whisper`) are installed and working.
3. **AI Integration:** If you have access, connect to Groq's LLM or a local Llama model to generate feedback. If time is too short or hardware is an issue, you can **simulate the AI outputs** with canned responses or a smaller model, but plan to explain how the real model would fit. (For instance: "Here we show a placeholder critique, but Groq+Llama will generate actual feedback.")
4. **Frontend and UI (Day 2):** Use Streamlit as planned for a quick UI. Create a sidebar or menu for the three modes. Add clear buttons ("Next Question", "Submit Explanation", "Analyze Code") and display areas for conversation. Integrate the Whisper STT so that clicking "Record" captures and transcribes speech. Test one flow end-to-end.
5. **Testing & Polish:** Debug any crashes. Simplify language in prompts so that AI answers are on-point. Make sure the app runs locally (if judges run it) or just works in your demo. Prepare one or two example sessions to showcase (e.g., answer "Two Sum" problem, then show code roast on a sample snippet).
6. **Finalize Demo Materials:** Capture screenshots of the app in use. Record a demo video (1-2 minutes) showing key features in action. Script it so you quickly explain the problem, show each mode briefly, and highlight results.

If you're working with a team, assign roles (e.g., one person on AI integration and backend, one on UI and UX, one on the presentation/video and write-up) to parallelize work.

Pitch and Demo Strategy

Your presentation (Devpost write-up + video) is as important as the code. Tell a *story*: start with **"What problem are we solving?"** (e.g., students need realistic coding practice). Then **show the product**: "Our app is like a personal interview coach." Emphasize *up to three* main features **【31†L162-L170】** – for example, 1) mock interview Q&A, 2) coding problem explain-aloud, 3) instant code feedback. A tip from hackathon winners: **"boil it down to a simple statement and highlight 3 key features or less" 【31†L162-L170】** .

In the demo video, pick the strongest flow. For instance:

- **Clip 1:** User selects "Interview Mode" and answers a simple question (e.g., "Reverse a string"). The system shows a score or feedback.

- **Clip 2:** (Optional) Switch to “Think Aloud”, present a known DSA problem, record your explanation via microphone, and display the AI’s analysis of your reasoning.
- **Clip 3:** “Code Roast” – paste some buggy Python code and show how the app catches errors and offers suggestions (perhaps humorously).

Narrate this as you go: “Here we used Python’s AST to find bugs, then prompted the LLM: ‘Give this code a humorous roast but end with constructive fix.’” Keep it engaging but clear. Make sure to mention how **Python libraries** you used power each step (e.g. “We use **pyflakes** to catch errors and a Groq-accelerated LLM to critique them”).

Close by summarizing what’s next or how it teaches the user: e.g. “Overall, this tool helps learners improve by simulating real interviews and giving candid feedback.” And definitely include the required Devpost elements: a short write-up in your own words, screenshots, the code repository link, and the recording [\[2†L212-L221\]](#) .

Conclusion

In summary, **yes**, this idea *can* win if executed sharply. It ticks many hackathon boxes: a compelling problem, heavy Python/AI use, and a novel twist (code roasting). To maximize your chance: **deliver a working prototype**, keep explanations **very clear**, and tie everything to the criteria. Use the hackathon judging guide as your checklist [\[2†L223-L225\]](#) [\[10†L281-L289\]](#) . If time is too tight to finish all modes, pivot to a simpler core and describe the rest conceptually. Either way, craft your demo to highlight Python implementation and the unique learning experience. Good luck!

Sources: Hackathon rules and criteria [\[1†L105-L110\]](#) [\[2†L287-L295\]](#) [\[2†L223-L225\]](#) ; similar project examples [\[24†L170-L173\]](#) [\[27†L199-L204\]](#) ; pitch advice [\[10†L281-L289\]](#) [\[31†L162-L170\]](#) .
